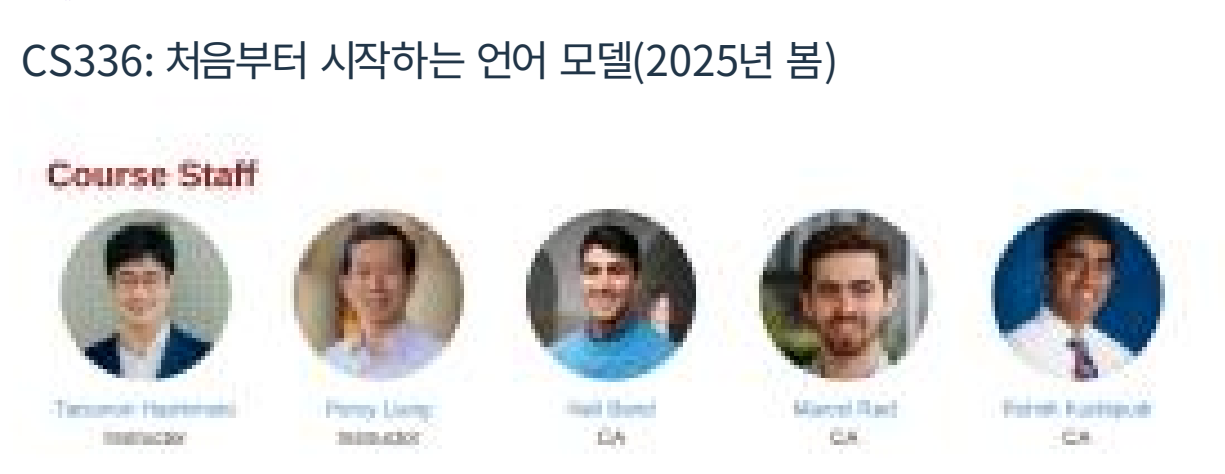
Machine Trans

강의_01.py

```
정규식 가져오기
abc 에서 ABC 가져오기
dataclasses 에서 data 가져오기
컬렉션 에서 defaultdic 져오기
개의 임의 가져오기

execute_util 에서 링크, 이미지, 텍스트를 가져옵니다 .
lecture_util 에서 article_link, x_link, youtube_link를 가져옵니다 .
참조 에서 gpt_3, gpt4, shannon1950, bengio2003, susketver2014, \를 가져옵니다 .
    bahdanau2015_주의, transformer_2017, gpt2, t5, kaplan_scaling_laws_2020, \
    the_pile, gpt_j, opt_175b, 블룸, 팜, 친칠라, 라마, 미스트랄_7b, \
    instruct_gpt, dpo, adamw2017, five, deepseek_v3, adam2014, grpo, ppo2017, muon, \
    대규모 배치 훈련_2018, wsd_2024, 코사인 학습_속도_2017, olmo_7b, moe_2017, \
    megatron_lm_2019, shazeer_2020, elmo, bert, qwen_2_5, deepseek_r1, moe_2017, \
    rms_norm_2019, rope_2021, 비누, gqa, mla, deepseek_67b, deepseek_v2, brants2007, \
    layernorm_2016, pre_post_norm_2020, llama2, llama3, olmo2, \
    메가바이트, BYTE5, BLT, TFREE, sennrich_2016, Zero_2019, GPipe_2018
데이터 가져 오기 get_common_crawl_urls, read_common_crawl, write_documents, markdownify_documents
model_util 에서 query_gpt4o 가져오기

틱톡 수입

def main():
    환영()
    이 과정이 존재하는 이유()
    현재 풍경()

    이 프로그램은 무엇인가요()

    과정_물류()
    코스_구성요소()

    토큰화()

    다음 시간: PyTorch 구성 요소, 리소스 회계


def welcome():
```

CS336: 처음부터 시작하는 언어 모델(2025년 봄)



Course Staff
Tatsunori Hashimoto
Instructor
Percy Liang
Instructor
Neil Band
CA
Marcel Rød
CA
Rohith Kuditipudi
CA

이것은 CS336의 두 번째 제공입니다.
스탠포드 에디션은 50% 성장했습니다.
강의는 YouTube에 게시되어 전 세계 사람들이 볼 수 있게 됩니다.

`def why_this_course_exists():`

왜 이 과정을 만들었나요?

GPT-4에 질문해 보겠습니 [오픈AI+ 2023]
다. response = query_gpt4o(prompt=" 언어 모델을 처음부터 구축하는 과정을 가르치는 이유는 무엇입니까? 한 문장으로 대답해 주세요.") # @inspect response

문제점: 연구자들이 기반 기술과 점점 단절되고 있습니다.
8년 전만 해도 연구자들은 자신만의 모델을 구현하고 훈련했습니다.
6년 전만 해도 연구자들은 모델(예: BERT)을 다운로드하고 미세 조정했습니다.
오늘날 연구자들은 독점적인 모델(예: GPT-4/Claude/Gemini)만을 개발하고 있습니다.

추상화 수준을 높이면 생산성이 향상되지만
- 이러한 추상화는 (프로그래밍 언어나 운영 체제와 달리) 누출되기 쉽습니다.
- 아직 스택을 분해해야 하는 근본적인 연구가 필요합니다.

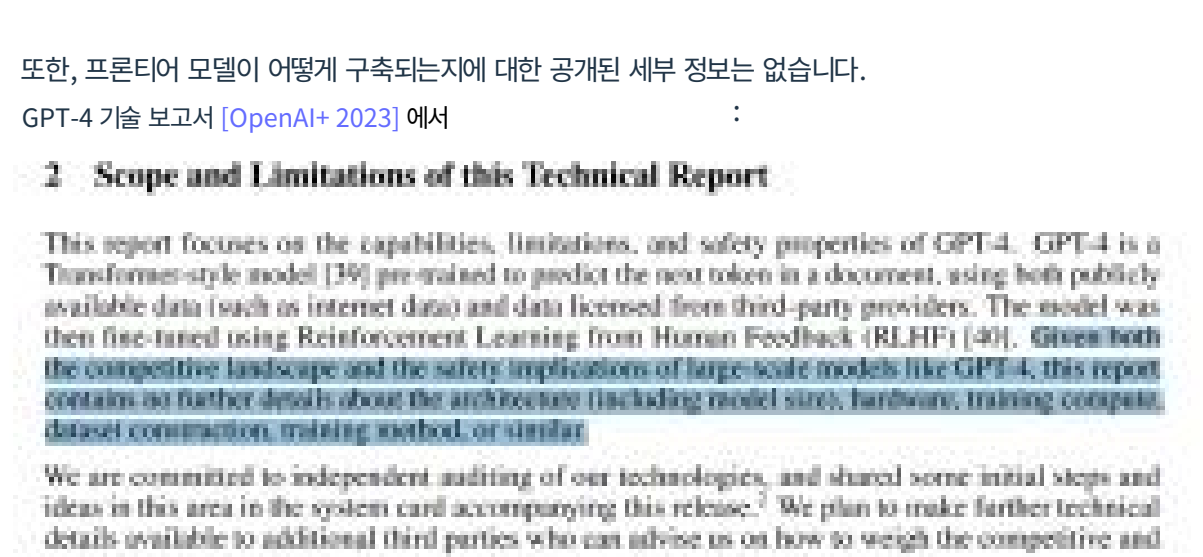
기초 연구를 위해서는 이 기술에 대한 완전한 이해가 필요합니다.

이 과정: 구축을 통한 이해
하지만 작은 문제가 하나 있는데요...

언어 모델의 산업화

GPT-4는 1.8T 매개변수를 가지고 있다고 합니다. [기사]
GPT-4를 훈련하는 데 1억 달러가 들었다고 합니다. [기사]
xAI는 Grok을 훈련시키기 위해 20만 대의 H100으로 클러스터를 구축했습니다. [기사]
스타게이트(OpenAI, NVIDIA, Oracle)가 4년간 5,000억 달러를 투자합니다. [기사]

또한, 프론티어 모델이 어떻게 구축되는지에 대한 공개된 세부 정보는 없습니다.
GPT-4 기술 보고서 [OpenAI+ 2023] 에서 :



2 Scope and Limitations of this Technical Report

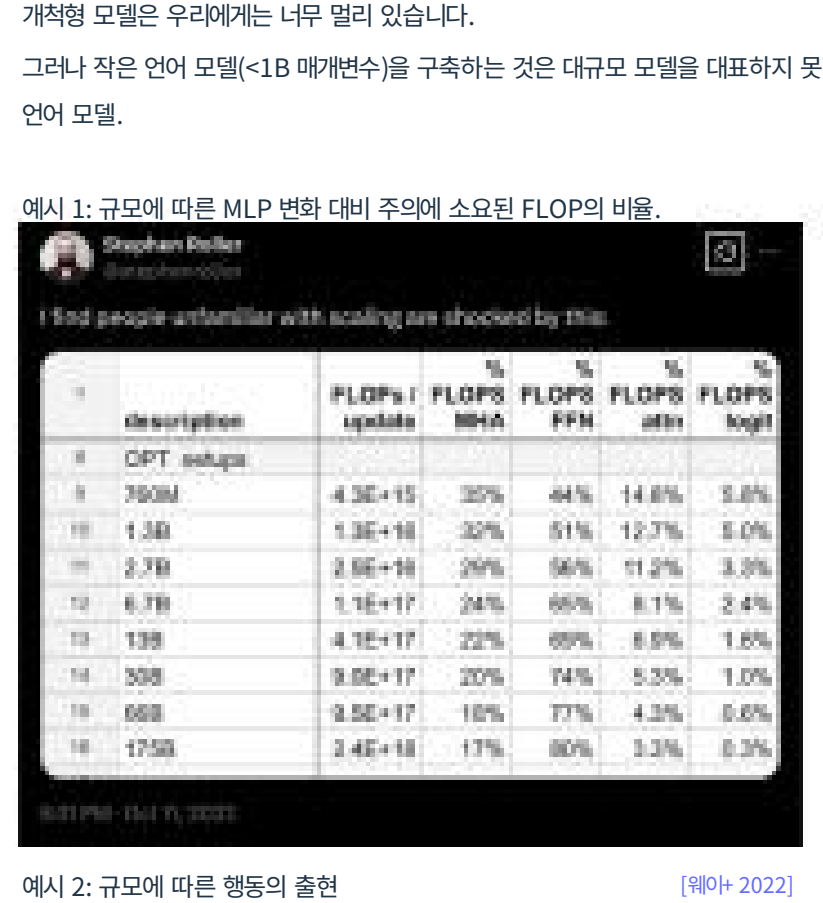
This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.[2] We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

더 많은 것이 다릅니다

개척형 모델은 우리에게는 너무 멀리 있습니다.
그러나 작은 언어 모델(<1B 매개변수)을 구축하는 것은 대규모 모델을 대표하지 못할 수 있습니다.
언어 모델.

예시 1: 규모에 따른 MLP 변화 대비 주의에 소요된 FLOP의 비율. [엑스]



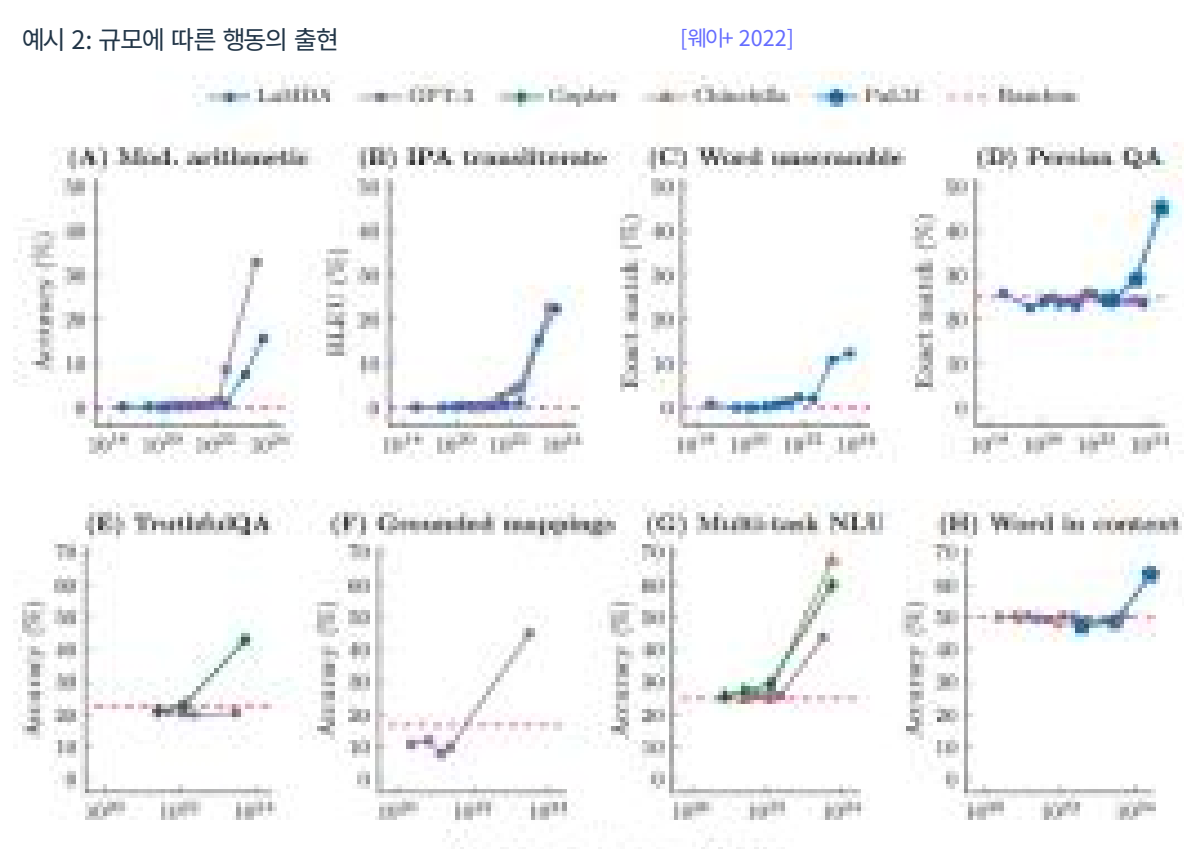
Stephen Roller
@stephenroller

I find people unfamiliar with scaling are shocked by this:

#	description	FLOPs / update	% FLOPS MHA	% FLOPS FFN	% FLOPS attn	% FLOPS logit
8	OPT setups					
9	760M	4.3E+15	35%	44%	14.8%	5.8%
10	1.3B	1.3E+16	32%	51%	12.7%	5.0%
11	2.7B	2.5E+16	29%	56%	11.2%	3.3%
12	6.7B	1.1E+17	24%	65%	8.1%	2.4%
13	13B	4.1E+17	22%	69%	6.9%	1.6%
14	30B	9.0E+17	20%	74%	5.3%	1.0%
15	66B	9.5E+17	18%	77%	4.3%	0.6%
16	175B	2.4E+18	17%	80%	3.3%	0.3%

6:31 PM · Oct 11, 2022

예시 2: 규모에 따른 행동의 출현 [웨이+ 2022]



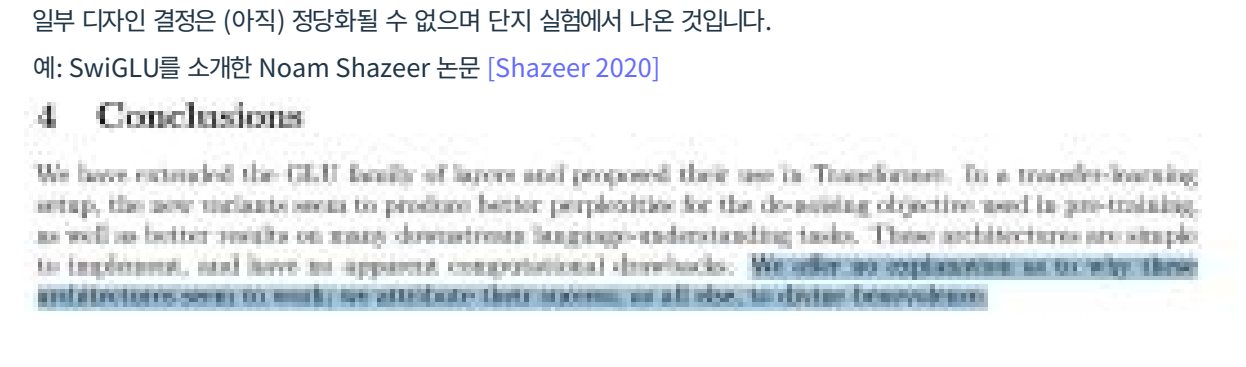
이 수업에서 프런티어 모델에 적용할 수 있는 어떤 내용을 배울 수 있나요?

지식에는 세 가지 유형이 있습니다.
- 역학: 사물이 작동하는 방식(변압기가 무엇인지, 모델 병렬 처리가 GPU를 활용하는 방식)
- 사고방식: 하드웨어를 최대한 활용하고 규모를 진지하게 받아들이는 것(규모 법칙)
- 직관: 어떤 데이터와 모델링 결정이 좋은 정확도를 제공하는가

우리는 기계적인 부분과 사고방식을 가르칠 수 있습니다(이것들은 전달됩니다).
우리는 직관을 부분적으로만 가르칠 수 있습니다(반드시 모든 척도를 넘나들 필요는 없습니다).

직감?

일부 디자인 결정은 (아직) 정당화될 수 없으며 단지 실험에서 나온 것입니다.
예: SwiGLU를 소개한 Noam Shazeer 논문 [Shazeer 2020]



4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learning setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-training, as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

쓰라린 교훈

잘못된 해석: 중요한 것은 규모이고 알고리즘은 중요하지 않습니다.
올바른 해석: 확장 가능한 알고리즘이 중요합니다.

정확도 = 효율성 x 리소스

사실, 더 큰 규모일수록 효율성이 훨씬 더 중요합니다(낭비를 감당할 수 없습니다).
[에르난데스+ 2020] 2012년부터 2019년까지 ImageNet에서 44배의 알고리즘 효율성을 보였습니다.

프레이밍: 특정 컴퓨팅 및 데이터 예산을 감안할 때 구축할 수 있는 가장 좋은 모델은 무엇입니까?
다시 말해, 효율성을 극대화하세요!

`def current_landscape():`

신경 이전(2010년대 이전)

- 영어의 엔트로피를 측정하는 언어 모델 [섀넌 1950]
- n-gram 언어 모델(기계 번역, 음성 인식)에 대한 많은 연구 [브랜츠+ 2007]

신경 성분(2010년대)

- 최초의 신경 언어 모델 [벤지오+ 2003]
- 시퀀스-투-시퀀스 모델링(기계 번역용) [수츠케버+ 2014]
- 아담 옵티마이저 [킹마+ 2014]
- 주의 메커니즘(기계 번역용) [바다나우+ 2014]
- 변압기 아키텍처(기계 번역용) [바스와니+ 2017]
- 전문가의 혼합 [샤지어+ 2017]
- 모델 병렬성 [황+ 2018][라즈반다리+ 2019][슈이비+ 2019]

초기 기초 모델(2010년대 후반)

- ELMo: LSTM을 이용한 사전 학습, 미세 조정으로 작업 수행에 도움 [피터스+ 2018]
- BERT: Transformer를 이용한 사전 학습, 미세 조정으로 작업 효율 향상 [데블린+ 2018]
- Google의 T5(11B): 모든 것을 텍스트 대 텍스트로 변환 [래플+ 2019]

확장성 수용, 더욱 폐쇄적

- OpenAI의 GPT-2(1.5B): 유창한 텍스트, 제로샷의 첫 징후, 단계적 출시 [래드포드+ 2019]
- 스케일링 법칙: 스케일링에 대한 희망/예측 가능성 제공 [카플란+ 2020]
- OpenAI의 GPT-3(175B): 컨텍스트 내 학습, 폐쇄형 [브라운+ 2020]
- Google의 PaLM(540B): 규모가 크고 훈련이 부족함 [차우드리+ 2022]
- DeepMind의 Chinchilla(70B): 최적의 계산 스케일링 법칙 [호프만+ 2022]

오픈 모델

- EleutherAI의 오픈 데이터 세트(The Pile) 및 모델(GPT-J) [가오+ 2020][왕+ 2021]
- Meta의 OPT(175B): GPT-3 복제, 많은 하드웨어 문제 [장+ 2022]
- 허깅 페이스 / BigScience의 BLOOM: 데이터 소싱에 집중 [워크숍+ 2022]
- 메타의 라마 모델 [투브롱+ 2023][투브롱+ 2023][그라타티오리+ 2024]
- 알리바바의 Qwen 모델 [큐웬+ 2024]
- DeepSeek의 모델 [DeepSeek-AI+ 2024][DeepSeek-AI+ 2024][DeepSeek-AI+ 2024]
- AI2의 OLMo 2 [그로네벨트+ 2024][OLMo+ 2024]

개방성 수준

- 폐쇄형 모델(예: GPT-4o): API 액세스만 가능 [오픈AI+ 2023]
- 개방형 가중치 모델(예: DeepSeek): 사용 가능한 가중치, 아키텍처 세부 정보가 포함된 논문, 일부 교육 세부 정보, 데이터 세부 정보 없음 [딥시크-AI+ 2024]
- 오픈 소스 모델(예: OLMo): 사용 가능한 가중치 및 데이터, 대부분의 세부 정보가 포함된 논문(하지만 반드시 그런 것은 아님)(이유, 실패한 실험) [그로네벨트+ 2024]

오늘날의 프론티어 모델

- OpenAI의 o3 https://openai.com/index/openai-o3-mini/
- 앤트로픽의 클로드 소네트 3.7 https://www.anthropic.com/news/claude-3-7-sonnet
- xAI의 Grok 3 https://x.ai/news/grok-3
- 구글의 제미니 2.5 https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-2025년 3월/
- 메타의 라마 3.3 https://ai.meta.com/blog/meta-llama-3/
- DeepSeek의 r1 [딥시크-AI+ 2025]
- 알리바바의 Qwen 2.5 Max https://qwenlm.github.io/blog/qwen2.5-max/
- 텐센트의 Hunyuan-T1 https://tencent.github.io/llm.hunyuan.T1/README_EN.html

`def what_is_this_program():`

이것은 실행 가능한 강의로, 강의 내용을 전달하는 프로그램입니다.
실행 가능한 강의를 통해 다음이 가능합니다.
- 코드를 보고 실행합니다(모든 것이 코드이기 때문에!)

```
총계 = 0 # @총계 검사
x가 [1, 2, 3] 인 경우 : # @x 검사
    총계 += x # @총계 검사
```

- 강의의 계층 구조를 보고,
- 정의 및 개념으로 이동: supervised_finetuning

`def course_logistics():`

모든 정보는 온라인에서 확인하세요: https://stanford-cs336.github.io/spring2025/

5개 과목으로 구성된 수업입니다.
2024년 봄학기 과정 평가에 대한 의견: 전체 과제는 대략 동일한 양이었습니다.
CS 224n의 5개 과제와 최종 프로젝트를 모두 완료해야 합니다. 그리고 이건 첫 번째 숙제일 뿐입니다.

이 과정을 수강해야 하는 이유

- 당신은 사물이 어떻게 작동하는지 이해하려는 강박관념을 가지고 있습니다.
- 당신은 연구 엔지니어링 역량을 키우고 싶어합니다.

이 과정을 수강해서는 안 되는 이유

- 실제로 이번 분기에는 연구를 하고 싶을 겁니다.
(상담사와 상담하세요.)
- 당신은 AI의 최신 기술(예: 멀티모달, RAG 등)에 관심이 있습니다.
(그것에 대해서는 세미나 수업을 들어야 합니다.)
- 당신은 자신의 애플리케이션 도메인에서 좋은 결과를 얻고 싶어합니다.
(기존 모델을 프롬프트하거나 미세 조정하면 됩니다.)

집에서 따라 할 수 있는 방법

- 모든 강의 자료와 과제는 온라인에 게시되므로, 자유롭게 스스로 따라오시기 바랍니다.
- 강의는 공식적으로 SCPD인 CGOE 를 통해 녹음됩니다 . YouTube에서 공개될 예정입니다(약간의 지연이 있을 수 있음).
- 우리는 내년에도 이 수업을 다시 제공할 계획입니다.

과제

- 5가지 과제(기본, 시스템, 확장 법칙, 데이터, 정렬).
- 스캐폴딩 코드는 없지만 정확성을 확인하는 데 도움이 되는 단위 테스트와 어댑터 인터페이스를 제공합니다.
- 정확성을 테스트하기 위해 로컬로 구현한 다음 벤치마킹(정확도 및 속도)을 위해 클러스터에서 실행합니다.
- 일부 과제에 대한 리더보드(훈련 예산을 고려하여 혼란을 최소화).
- AI 도구(예: CoPilot, Cursor)는 학습 효과를 떨어뜨릴 수 있으므로, 사용 시 모든 위험을 감수하시기 바랍니다.

무리

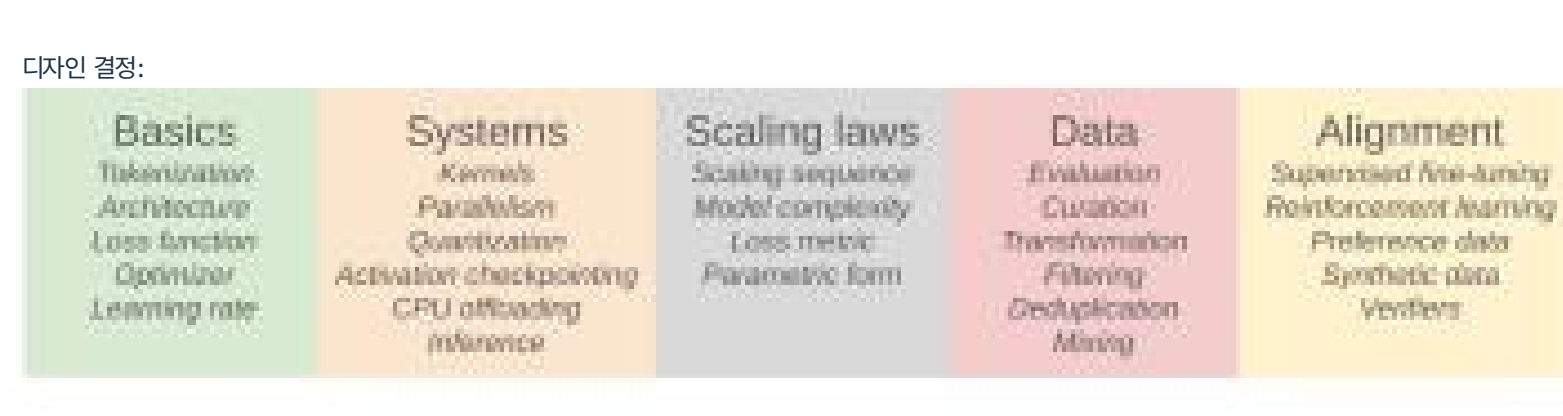
- 컴퓨팅 클러스터를 제공해 주신 Together AI에 감사드립니다.
- 가이드를 읽어주세요 클러스터를 사용하는 방법에 대해.
- 마감일이 가까워지면 클러스터가 꽉 차므로 일찍 과제를 시작하세요!

`def course_components():`

모든 것은 효율성에 관한 것입니다

리소스: 데이터 + 하드웨어(컴퓨팅, 메모리, 통신 대역폭)
고정된 리소스 세트가 주어졌을 때 어떻게 최상의 모델을 훈련시키나요?
예: 2주 동안 Common Crawl 덤프와 H100 32개가 주어지면 어떻게 해야 할까요?

디자인 결정:



Basics
Tokenization
Architecture
Loss function
Optimizer
Learning rate
Systems
Kernels
Parallelism
Quantization
Activation checkpointing
CPU offloading
Inference
Scaling laws
Scaling sequence
Model complexity
Loss metric
Parametric form
Data
Evaluation
Curation
Transformation
Filtering
Deduplication
Mixing
Alignment
Supervised fine-tuning
Reinforcement learning
Preference data
Synthetic data
Verifiers

과정 개요

```
기초()
시스템()
스케일링_법칙()
데이터()
조정()
```

효율성은 디자인 결정을 좌우합니다

오늘날 우리는 컴퓨팅에 제약을 받고 있으므로 설계 결정은 주어진 것을 최대한 활용하는 것을 반영하게 될 것입니다.
하드웨어.
- 데이터 처리: 불량/무관한 데이터에 귀중한 컴퓨팅 업데이트를 낭비하지 마십시오.
- 토큰화: 원시 바이트로 작업하는 것은 우아하지만 오늘날의 모델 아키텍처로는 계산 효율이 떨어집니다.
- 모델 아키텍처: 메모리 또는 FLOP를 줄이는 데 따른 많은 변경 사항(예: KV 캐시 공유, 슬라이딩 윈도우 주의)
- 훈련: 단일 에포크만으로도 충분해요!
- 스케일링 법칙: 하이퍼파라미터 튜닝을 위해 더 작은 모델에서 더 적은 컴퓨팅을 사용합니다.
- 정렬: 원하는 사용 사례에 맞게 모델을 조정하는 경우 더 작은 기본 모델이 필요합니다.

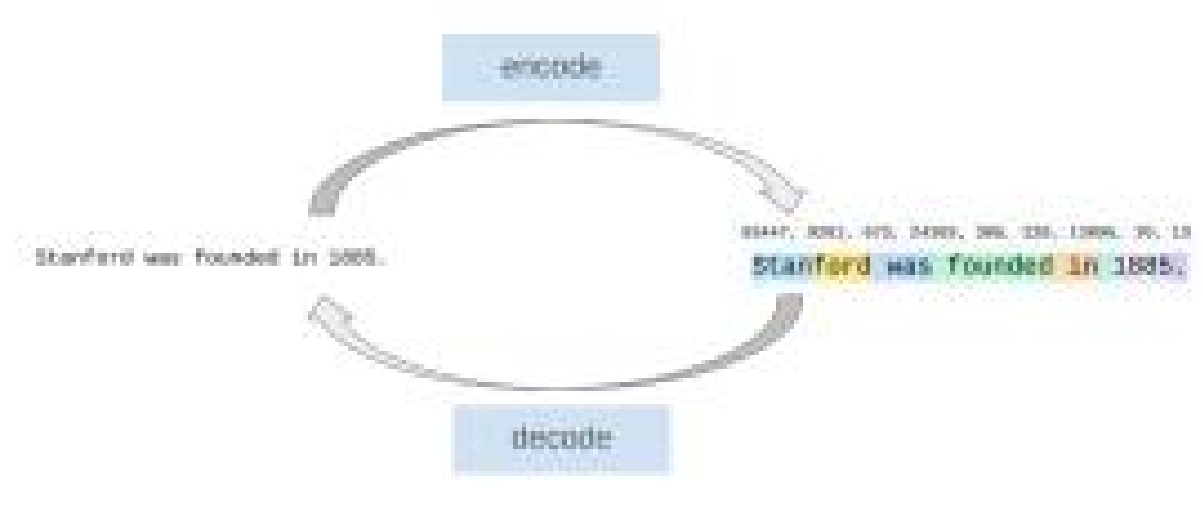
내일은 데이터가 제한적이 될 것입니다.

```
클래스 토크나이저(ABC):
    """토큰화기를 위한 추상 인터페이스."""
    def encode(self, string: str) -> list[int]:
        NotImplementedError를 발생시키다

    def decode(self, indices: list[int]) -> str:
        NotImplementedError를 발생시키다


def basics():
```

목표: 전체 파이프라인의 기본 버전을 작동시키는 것
구성 요소: 토큰화, 모델 아키텍처, 학습

토큰화

토크나이저는 문자열과 정수 시퀀스(토큰) 사이를 변환합니다.



encode
Stanford was founded in 1885.
Stanford was founded in 1885.
decode

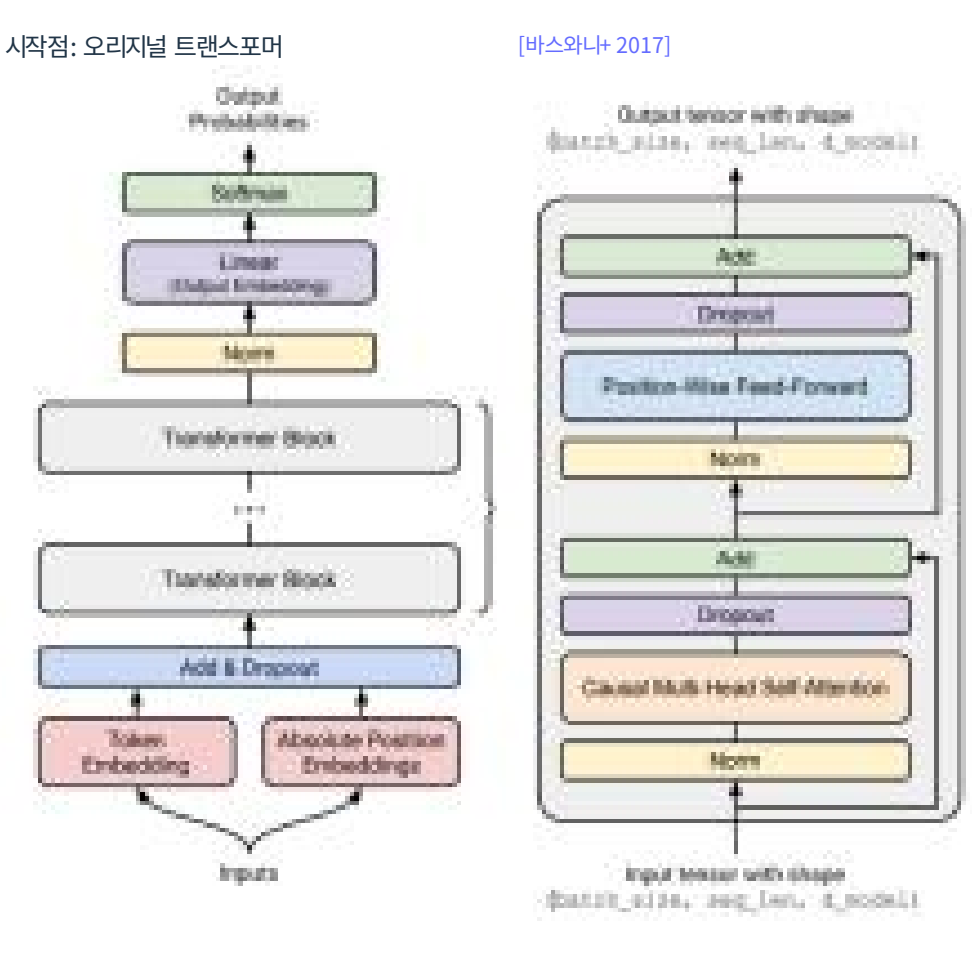
직관: 문자열을 인기 있는 세그먼트로 나누세요

이 과정: 바이트 쌍 인코딩(BPE) 토크나이저 [센리히+ 2015]

토크나이저가 없는 접근 방식: [Xue+ 2021] [Yu+ 2023] [Pagnoni+ 2024] [Deiseroth+
바이트를 직접 사용하면 전망은 밝지만 아직 경계까지 확장되지는 않았습니다.

건축학

시작점: 오리지널 트랜스포머 [바스와니+ 2017]



Output Probabilities
Softmax
Linear (Output Embedding)
Norm
Transformer Block
Transformer Block
Add & Dropout
Token Embedding
Absolute Position Embeddings
Inputs
Output tensor with shape (batch_size, seq_len, d_model)
Add
Dropout
Position-Wise Feed-Forward
Norm
Add
Dropout
Causal Multi-Head Self-Attention
Norm
Input tensor with shape (batch_size, seq_len, d_model)

변형:
- 활성화 함수: ReLU, SwiGLU [샤지르 2020]
- 위치 인코딩: 사인파, RoPE [2021년 일+]
- 정규화: LayerNorm, RMSNorm [Ba+ 2016][장+ 2019]
- 정규화 배치: 사전 정규화 대 사후 정규화 [숑+ 2020]
- MLP: 전문가의 혼합, 밀도 있는 [샤지어+ 2017]
- 주의: 전체, 슬라이딩 윈도우, 선형 [장+ 2023][카타로폴로스+ 2020]
- 저차원 주의: 그룹 쿼리 주의(GQA), 다중 헤드 잠재 주의(MLA) [에인슬리+ 2023][DeepSeek-AI+ 2024]
- 상태 공간 모델: 하이에나 [폴리+ 2023]

훈련

- 최적화 프로그램(예: AdamW, Muon, SOAP) [킹마+ 2014][로스칠로프+ 2017][켈러 2024][비아스+ 2024]
- 학습률 일정(예: 코사인, WSD) [로쉴로프+ 2016][후+ 2024]
- 배치 크기(예: 중요 배치 크기) [맥캔들리시+ 2018]
- 정규화(예: 드롭아웃, 가중치 감소)
- 하이퍼파라미터(헤드 수, 숨겨진 차원): 그리드 검색

과제 1

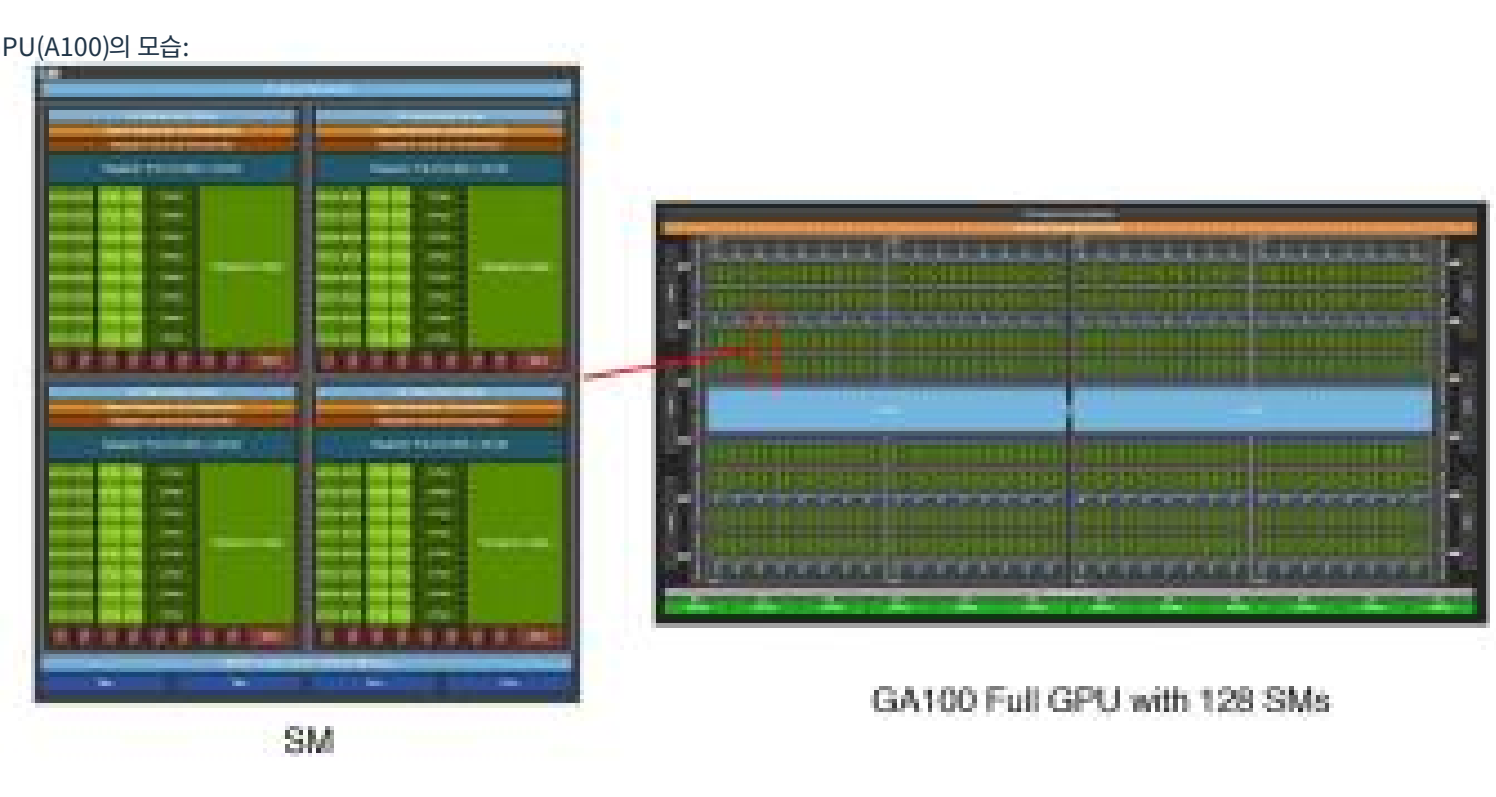
[GitHub][PDF]
- BPE 토크나이저 구현
- Transformer, Cross-Entropy Loss, AdamW Optimizer, Training Loop 구현
- TinyStories 및 OpenWebText에서 교육
- 리더보드: H100에서 90분 동안 OpenWebText의 복잡성을 최소화하세요 [작년 순위표]

`def 시스템():`

목표: 하드웨어를 최대한 활용
구성 요소: 커널, 병렬성, 추론

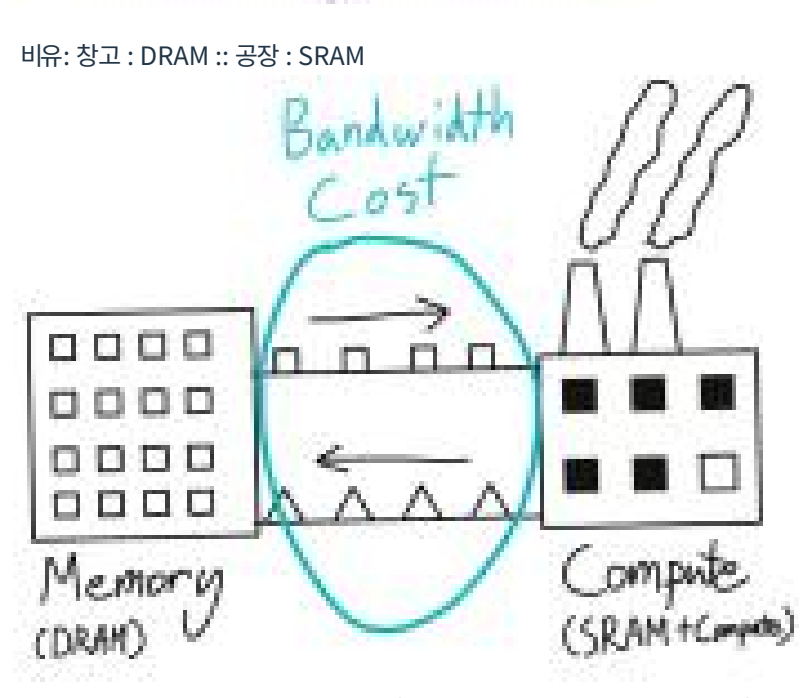
커널

GPU(A100)의 모습:



SM
GA100 Full GPU with 128 SMs

비유: 창고 : DRAM :: 공장 : SRAM

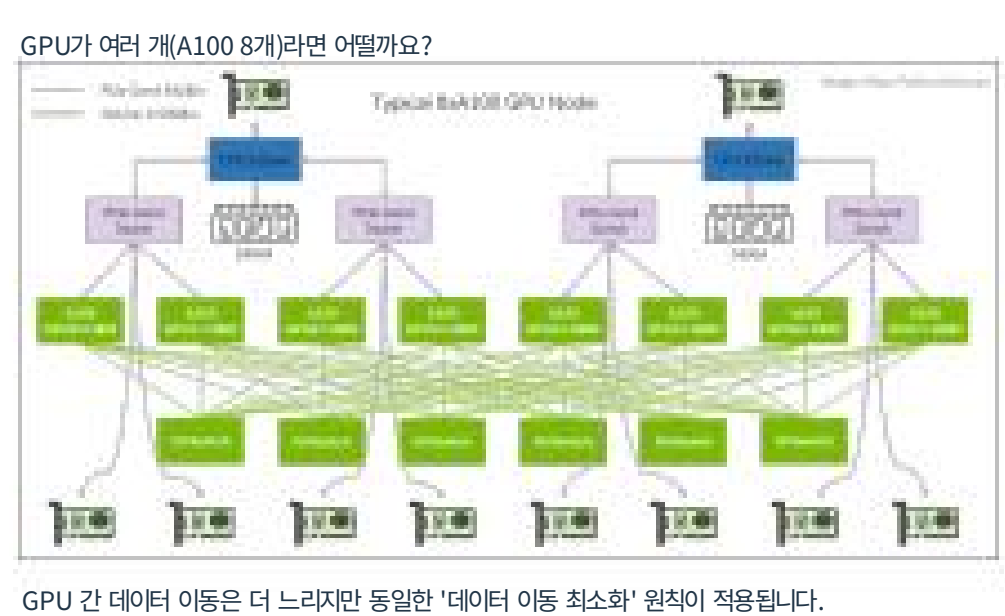


Bandwidth Cost
Memory (DRAM)
Compute (SRAM + Compute)

요령: 데이터 이동을 최소화하여 GPU 활용도를 극대화하도록 계산을 구성합니다.
CUDA/Triton/CUTLASS/ThunderKittens에서 커널을 작성하세요

병행

GPU가 여러 개(A100 8개)라면 어떨까요?

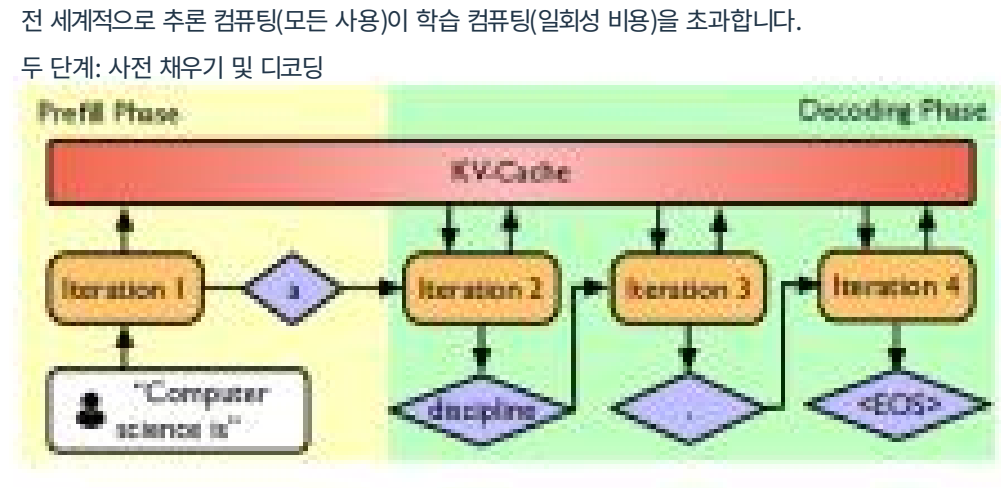


Typical 8xA100 GPU Node

GPU 간 데이터 이동은 더 느리지만 동일한 '데이터 이동 최소화' 원칙이 적용됩니다.
집합적 작업 사용(예: 수집, 축소, 전체 축소)
GPU 간 샤드(매개변수, 활성화, 그래디언트, 최적화 상태)
계산을 분할하는 방법: {data,tensor,pipeline,sequence} 병렬 처리

추론

목표: 프롬프트가 주어졌을 때 토큰을 생성합니다(실제로 모델을 사용하는 데 필요함!)
강화 학습, 테스트 시간 컴퓨팅, 평가에도 추론이 필요합니다.
전 세계적으로 추론 컴퓨팅(모든 사용)이 학습 컴퓨팅(일회성 비용)을 초과합니다.
두 단계: 사전 채우기 및 디코딩



Prefill Phase
Decoding Phase
KV-Cache
Iteration 1
a
Iteration 2
Iteration 3
Iteration 4
"Computer science is"
discipline
.
<EOS>

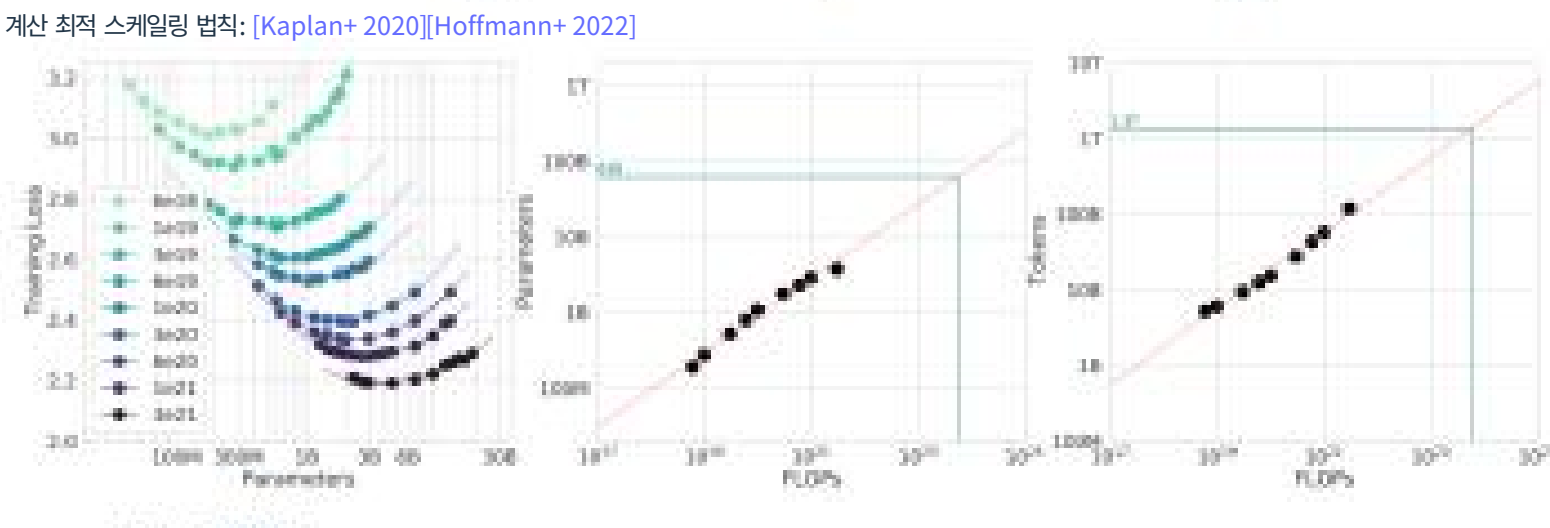
사전 채우기(훈련과 유사): 토큰이 제공되고 모든 것을 한 번에 처리할 수 있습니다(컴퓨팅 바운드).
디코드: 한 번에 하나의 토큰을 생성해야 함(메모리 제한)
디코딩 속도를 높이는 방법:
- 더 저렴한 모델 사용(모델 정리, 양자화, 증류를 통해)
- 추측 디코딩: 더 저렴한 "초안" 모델을 사용하여 여러 토큰을 생성한 다음 전체 모델을 사용하여 병렬로 점수를 매기세요(정확한 디코딩!)
- 시스템 최적화: KV 캐싱, 배칭

과제 2

[2024년부터 GitHub][2024년부터 PDF]
- Triton에서 융합된 RMSNorm 커널 구현
- 분산 데이터 병렬 학습 구현
- 최적화기 상태 분할 구현
- 구현 벤치마크 및 프로파일링

`def 스케일링_법칙():`

목표: 소규모 실험 수행, 대규모 하이퍼파라미터/손실 예측
질문: FLOP 예산(C)이 주어졌을 때 더 큰 모델(N)을 사용하거나 더 많은 토큰(D)으로 훈련해야 할까요?
계산 최적 스케일링 법칙: [Kaplan+ 2020][Hoffmann+ 2022]



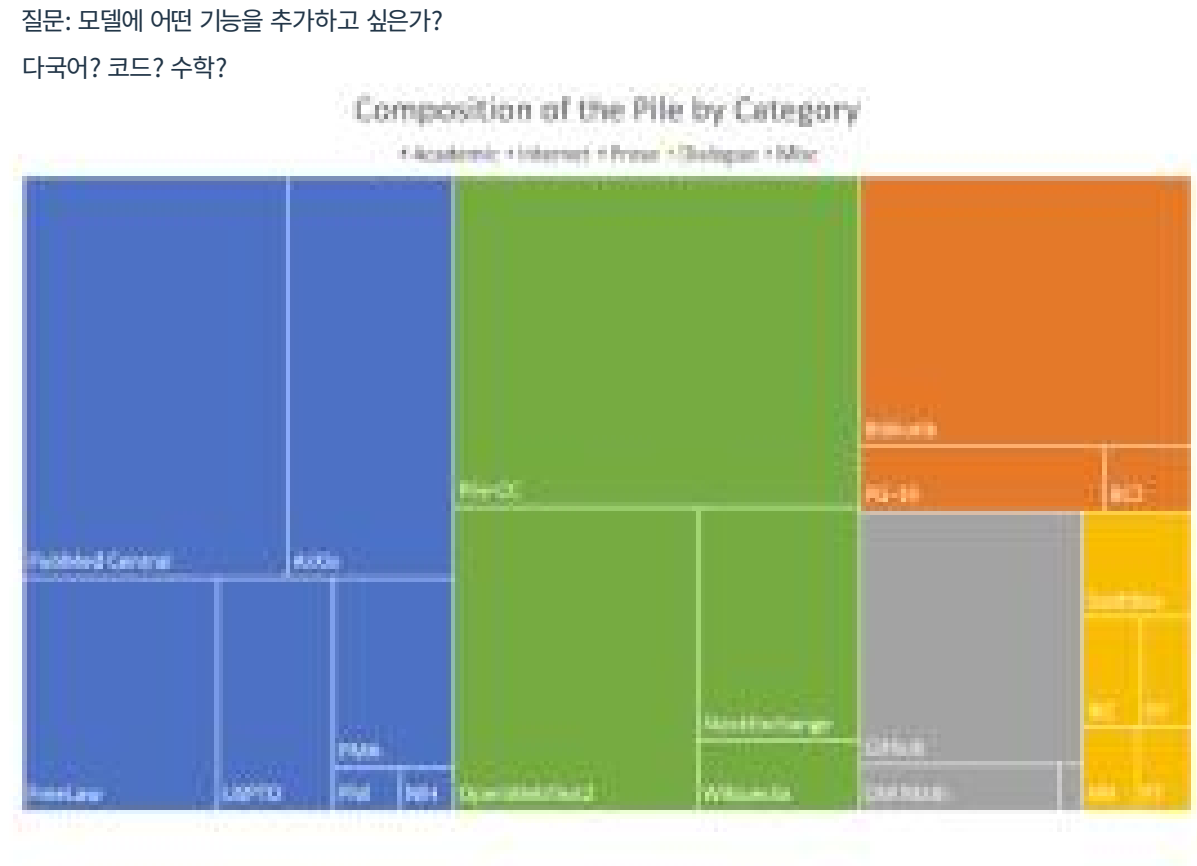
요약: 하 $D^* = 20N^*$ (예: 1.4B 매개변수 모델은 28B 토큰에서 학습되어야 함)
지만 여기에는 추론 비용이 고려되지 않았습니다!

과제 3

[2024년부터 GitHub][2024년부터 PDF]
- 이전 실행을 기반으로 훈련 API(하이퍼매개변수 -> 손실)를 정의합니다.
- "훈련 작업"을 제출하고(FLOP 예산 내에서) 데이터 포인트를 수집합니다.
- 데이터 포인트에 스케일링 법칙을 적용합니다.
- 확장된 하이퍼매개변수에 대한 예측 제출
- 리더보드: FLOP 예산을 고려한 손실 최소화

`def date():`

질문: 모델에 어떤 기능을 추가하고 싶은가?
다국어? 코드? 수학?



평가

- Perplexity: 언어 모델을 위한 교과서 평가
- 표준화된 테스트(예: MMLU, HellaSwag, GSM8K)
- 지침 따르기(예: AlpacaEval, IFEval, WildBench)
- 테스트 시간 컴퓨팅 확장: 사고의 사슬, 앙상블
- LM-as-a-judge: 생성 작업 평가
- 전체 시스템: RAG, 에이전트

데이터 큐레이션

- 데이터는 하늘에서 갑자기 떨어지는 것이 아닙니다.
웹데이터를 살펴보세요()
- 출처: 인터넷에서 크롤링한 웹페이지, 서적, arXiv 논문, GitHub 코드 등
- 저작권 데이터에 대한 교육을 위해 공정 사용에 호소하는가? [헨더슨+ 2023]
- 데이터 라이선스가 필요할 수 있음(예: Reddit 데이터를 사용하는 Google) [기사]
- 형식: HTML, PDF, 디렉토리(텍스트 제외!)

데이터 처리

- 변환: HTML/PDF를 텍스트로 변환(내용, 일부 구조, 재작성 유지)
- 필터링: 고품질 데이터 유지, 유해 콘텐츠 제거(분류기를 통해)
- 중복 제거: 컴퓨팅을 절약하고 기억을 방지합니다. Bloom 필터 또는 MinHash를 사용합니다.

과제 4

[2024년부터 GitHub][2024년부터 PDF]
- 일반 크롤링 HTML을 텍스트로 변환
- 품질이 좋고 유해한 콘텐츠를 필터링하기 위한 분류기 훈련
- MinHash를 사용한 중복 제거
- 리더보드: 토큰 예산을 고려하여 혼란을 최소화합니다.

```
def look_at_web_data():
    urls = get_common_crawl_urls()[:3] # @urls 검사
    문서 = list(read_common_crawl(urls[1], limit=300))
    랜덤.시드(40)
    random.shuffle(문서)
    문서 = markdownify_documents(문서[:10])
    write_documents(문서, "var/sample-documents.txt")
```

[샘플 문서]
정말 황무지네요! 데이터 처리가 정말 필요해요.

`def 정렬():`

지금까지 기본 모델은 잠재력이 매우 뛰어나며 다음 토큰을 완성하는 데 매우 좋습니다.
정렬을 통해 모델이 실제로 유용해집니다.

정렬의 목표:
- 지침을 따르기 위한 언어 모델을 얻으세요
- 스타일(형식, 길이, 톤 등)을 조정합니다.
- 안전을 통합하세요(예: 해로운 질문에 대답하기를 거부하는 것)

두 단계:
감독 미세 조정()
피드백으로부터의 학습()

Machine Translated by Google

과제 5

[2024년부터 GitHub][2024년부터 PDF]

- 감독된 미세 조정 구현
- 직접 선호도 최적화(DPO) 구현
- 그룹 상대 선호도 최적화(GRPO) 구현

```
@dataclass(동결=참)
클래스 턴:
    역할: str
    내용: str


@dataclass(동결=참)
클래스 채팅 예:
    차례: list[차례]


@dataclass(동결=참)
클래스 선호도 예:
    역사: 목록[회전]
    response_a: 문자열
    응답_b: 문자열
    선택됨: str


def supervised_finetuning():
```

감독 미세 조정(SFT)

지시 데이터: (프롬프트, 응답) 쌍

```
    sft_data: 목록[채팅예제] = [
        채팅예제(
            회전=[
                Turn(role="system", content="당신은 도움이 되는 조수입니다.")
                Turn(role="user", content=" 1 + 1은 무엇입니까?"),
                Turn(role="assistant", content=" 정답은 2입니다."),
            ],
        ),
    ]
```

데이터에는 종종 사람의 주석이 포함됩니다.

직관: 기본 모델에는 이미 해당 기술이 포함되어 있으므로 표현하기 위해 몇 가지 예만 필요합니다. [저우+ 2023]

지도 학습: p(응답 | 프롬프트)를 최대화하기 위해 모델을 미세 조정합니다.

```
def 학습_피드백():
```

이제 우리는 모델을 따르는 예비 지침을 갖게 되었습니다.

값비싼 주석 없이도 더 나은 결과를 만들어 보겠습니다.

선호도 데이터

데이터: 주어진 프롬프트에 대한 모델(예: [A, B])을 사용하여 여러 응답을 생성합니다.

사용자가 기본 설정을 제공합니다(예: A < B 또는 A > B).

```
    선호도_데이터: 목록[선호도 예시] = [
        선호도 예시(
            역사=[
                Turn(role="system", content="당신은 도움이 되는 조수입니다.")
                Turn(role="user", content=" 언어 모델을 훈련하는 가장 좋은 방법은 무엇입니까?"),
            ],
            response_a=" 대용량 데이터 세트를 사용하고 장시간 학습해야 합니다.",
            response_b=" 작은 데이터 세트를 사용하고 짧은 시간 동안 학습해야 합니다.",
            선택됨="a",
        )
    ]
```

검증자

- 공식 검증자(예: 코드, 수학)
- 학습된 검증자: LM-as-a-judge에 대한 훈련

알고리즘

- 강화 학습을 통한 근접 정책 최적화(PPO) [슐만+ 2017][오우양+ 2022]
- 직접 정책 최적화(DPO): 선호도 데이터의 경우 더 간단함 [라파엘로프+ 2023]
- 그룹 상대 선호도 최적화(GRPO): 가치 함수 제거 [샤오+ 2024]

```
############################################################
# 토큰화

# https://github.com/openai/tiktoken/blob/main/tiktoken_ext/openai_public.py#L23
GPT2_토크나이저_정규식 = \
    r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""


def 토큰화():
```

이 단원은 Andrej Karpathy의 토큰화에 관한 영상에서 영감을 얻었습니다. 확인해 보세요! [동영상]

```
    토큰화 소개()
    토큰화_예제()
    캐릭터_토큰화()
    바이트_토큰화기()
    단어_토큰화()
    bpe_tokenizer()
```

요약

- 토크나이저: 문자열 <-> 토큰(인덱스)
- 문자 기반, 바이트 기반, 단어 기반 토큰화는 매우 최적이 아닙니다.
- BPE는 코퍼스 통계를 살펴보는 효과적인 휴리스틱입니다.
- 토큰화는 불가피한 일인데, 어쩌면 언젠가는 바이트 단위로 처리할 수도 있을 것 같습니다.

```
@dataclass(동결=참)
클래스 BPETokenizerParams:
    """BPETokenizer를 지정하는 데 필요한 모든 것."""
    어휘: dict[int, bytes] # 인덱스 -> 바이트
    병합: dict[tuple[int, int], int] # index1,index2 -> new_index



클래스 CharacterTokenizer(토큰화기):
    """문자열을 유니코드 코드 포인트의 시퀀스로 나타냅니다."""
    def encode(self, string: str) -> list[int]:
        list(map(ord, string)) 를 반환합니다 .

    def decode(self, indices: list[int]) -> str:
        "".join(map(chr, indices)) 을 반환합니다 .


클래스 ByteTokenizer(토큰화기):
    """문자열을 바이트 시퀀스로 표현합니다."""
    def encode(self, string: str) -> list[int]:
        string_bytes = string.encode("utf-8") # @inspect string_bytes
        인덱스 = list(map(int, string_bytes)) # @inspect 인덱스
        반환 인덱스

    def decode(self, indices: list[int]) -> str:
        string_bytes = bytes(인덱스) # @inspect string_bytes
        문자열 = string_bytes.decode("utf-8") # @inspect 문자열
        반환 문자열


def merge(인덱스: 리스트[int], 페어: 튜플[int, int], new_index: int) -> 리스트[int]: # @inspect 인덱스, @inspect 페어, @inspect new_index
    """`pair`의 모든 인스턴스를 `new_index`로 바꿔 `indices`를 반환합니다."""
    새로운 인덱스 = [] # @inspect 새로운 인덱스
    i = 0 # @검사 i
    i < len(인덱스) 인 경우 :
        i + 1 < len(indices) 이고 indices[i] == pair[0] 이고 indices[i + 1] == pair[1]인 경우 :
            새로운 인덱스.추가(새로운 인덱스)
            i += 2
        또 다른:
            새로운 인덱스.추가(인덱스[i])
            i += 1
    new_indices를 반환합니다


클래스 BPETokenizer(토큰화기):
    """BPE 토크나이저에 병합 세트와 어휘가 제공됩니다."""
    def __init__(self, 매개변수: BPETokenizerParams):
        self.params = 매개변수

    def encode(self, string: str) -> list[int]:
        인덱스 = list(map(int, string.encode("utf-8"))) # @inspect 인덱스
        # 참고: 이것은 매우 느린 구현입니다.
        쌍 의 경우 self.params.merges.items() 의 new_index : # @inspect 쌍, @inspect new_index
            인덱스 = merge(인덱스, 쌍, 새 인덱스)
        반환 인덱스

    def decode(self, indices: list[int]) -> str:
        bytes_list = list(map(self.params.vocab.get, indices)) # @inspect bytes_list
        문자열 = b"".join(bytes_list).decode("utf-8") # @inspect 문자열
        반환 문자열


def get_compression_ratio(문자열: str, 인덱스: list[int]) -> float:
    """`문자열`이 `인덱스`로 토큰화되면, ."""
    num_bytes = len(bytes(문자열, 인코딩="utf-8")) # @inspect num_bytes
    num_tokens = len(인덱스)                                    # @inspect 토큰 수
    num_bytes / num_tokens를 반환합니다 .


def get_gpt2_tokenizer():
    # 코드: https://github.com/openai/tiktoken
    # gpt3.5-turbo 또는 gpt4 토크나이저에는 cl100k_base를 사용할 수 있습니다.
    tiktoken.get_encoding("gpt2")을 반환합니다 .


def intro_to_tokenization():
```

원시 텍스트는 일반적으로 유니코드 문자열로 표현됩니다.

```
    문자열 = "안녕하세요,       🌍 ! 안녕하세요!"
```

언어 모델은 토큰 시퀀스(일반적으로 정수로 표현됨)에 대한 확률 분포를 배치합니다.
지수).

```
    인덱스 = [15496, 11, 995, 0]
```

그래서 문자열을 토큰으로 인코딩하는 절차가 필요합니다 .
또한 토큰을 다시 문자열로 디코딩하는 절차도 필요합니다 .
토크 나이저 인코딩 및 디코딩 메서드를 구현하는 클래스입니다.
어휘 크기는 가능한 토큰(정수)의 수입니다.

```
def 토큰화_예제():
```

토크나이저가 어떻게 작동하는지 알아보려면 이것으로 놀아보세요. 대화형 사이트

관찰

- 단어와 그 앞의 공백은 같은 토큰의 일부입니다(예: "world").
- 처음과 중간에 있는 단어는 다르게 표현됩니다(예: "안녕하세요, 안녕하세요").
- 숫자는 몇 자리 단위로 토큰화됩니다.

OpenAI(tiktoken)의 GPT-2 토크나이저가 실제로 작동하는 모습입니다.

```
    토크나이저 = get_gpt2_tokenizer()
    문자열 = "안녕하세요,       🌍 ! 안녕하세요! # @inspect 문자열
```

encode() 및 decode() 왕복을 확인해보세요.

```
    인덱스 = tokenizer.encode(문자열) # @inspect 인덱스
    재구성된 문자열 = tokenizer.decode(인덱스) # @inspect 재구성된 문자열
    문자열 == 재구성된 문자열을 주장합니다 .
    압축 비율 = get_compression_ratio(문자열, 인덱스) # @inspect 압축 비율


def character_tokenizer():
```

문자 기반 토큰화

유니코드 문자열은 유니코드 문자의 시퀀스입니다.

각 문자는 ord를 통해 코드 포인트(정수)로 변환될 수 있습니다 .

```
    ord("a") == 97을 주장합니다 .
    ord(" ") == 127757🌍합니다 .
```

chr 을 통해 다시 변환할 수 있습니다 .

```
    chr(97) == "a"를 주장합니다 .
    chr(127757)을 주장하다 ==        🌍
```

이제 Tokenizer 를 빌드 하고 왕복이 가능한지 확인해 보겠습니다.

```
    토크나이저 = CharacterTokenizer()
    문자열 = "안녕하세요, 인덱스   🌍 ! 안녕하세요! # @inspect 문자열
    = tokenizer.encode(문자열) # @inspect 인덱스
    재구성된 문자열 = tokenizer.decode(인덱스) # @inspect 재구성된 문자열
    문자열 == 재구성된 문자열을 주장합니다 .
```

유니코드 문자는 약 15만 개입니다. [위키피디아]

```
    vocabulary_size = max(indices) + 1 # 이것은 하한입니다 @inspect vocabulary_size
```

문제 1: 이것은 어휘가 매우 방대합니다.

문제 2: 많은 문자가 매우 드물기 때문에(예:) 어휘를 비효율적으로 사🌍니다.

```
    압축 비율 = get_compression_ratio(문자열, 인덱스) # @inspect 압축 비율


def byte_tokenizer():
```

바이트 기반 토큰화

유니코드 문자열은 0 사이의 정수로 표현될 수 있는 바이트 시퀀스로 표현될 수 있습니다.
그리고 255.

가장 일반적인 유니코드 인코딩은 다음과 같습니다. UTF-8

일부 유니코드 문자는 1바이트로 표현됩니다.

```
    바이트("a", 인코딩="utf-8") == b"a"를 주장합니다 .
```

다른 것들은 여러 바이트를 사용합니다.

```
    바이트(" ", 인코딩="utf-8🌍 b"\xf0\x9f\x8c\x8d"를 주장합니다 .
```

이제 Tokenizer 를 빌드 하고 왕복이 가능한지 확인해 보겠습니다.

```
    토크나이저 = ByteTokenizer()
    문자열 = "안녕하세요, 인덱스   🌍 ! 안녕하세요! # @inspect 문자열
    = tokenizer.encode(문자열) # @inspect 인덱스
    재구성된 문자열 = tokenizer.decode(인덱스) # @inspect 재구성된 문자열
    문자열 == 재구성된 문자열을 주장합니다 .
```

어휘는 훌륭하고 적습니다. 바이트는 256개의 값을 표현할 수 있습니다.

```
    어휘_크기 = 256 # @inspect 어휘_크기
```

압축률은 어떻습니까?

```
    압축 비율 = get_compression_ratio(문자열, 인덱스) # @inspect 압축 비율
    압축률 == 1을 주장합니다
```

압축률이 너무 낮아서 시퀀스가 너무 길어집니다.

Transformer의 컨텍스트 길이가 제한되어 있다는 점(주의가 2차적이기 때문에)을 고려하면 이는 다음과 같이 보이지 않습니다.
엄청난...

```
def word_tokenizer():
```

단어 기반 토큰화

또 다른 접근 방식(NLP에서 고전적으로 수행되던 방식에 더 가까움)은 문자열을 단어로 분할하는 것입니다.

```
    string = "저는 supercalifragilisticexpialidocious라고 말할게요!"

    세그먼트 = regex.findall(r"\w+|.", 문자열) # @inspect 세그먼트
```

이 정규 표현식은 모든 영숫자 문자(단어)를 함께 유지합니다.

좀 더 멋진 버전은 다음과 같습니다.

```
    패턴 = GPT2_TOKENIZER_REGEX # @패턴 검사
    세그먼트 = regex.findall(패턴, 문자열) # @inspect 세그먼트
```

이것을 토크나이저 로 바꾸려면 이러한 세그먼트를 정수로 매핑해야 합니다.

그러면 각 세그먼트에서 정수로의 매핑을 구축할 수 있습니다.

하지만 문제가 있습니다.

- 단어의 수가 엄청납니다(유니코드 문자의 경우처럼).
- 많은 단어는 희귀하기 때문에 모델은 그 단어에 대해 많은 것을 배울 수 없습니다.
- 이는 분명 고정된 어휘 규모를 제공하지 않습니다.

훈련 중에 본 적이 없는 새로운 단어는 보기 흉하고 혼란을 일으킬 수 있는 특수한 UNK 토큰을 받습니다.
계산.

```
    vocabulary_size = "훈련 데이터의 고유 세그먼트 수"
    압축 비율 = get_compression_ratio(문자열, 세그먼트) # @inspect 압축 비율


def bpe_tokenizer():
```

바이트 쌍 인코딩(BPE)

[위키피디아]

BPE 알고리즘은 1994년 필립 게이지가 데이터 압축을 위해 도입했습니다. [기사]

이는 신경망 기계 번역을 위해 NLP에 맞게 조정되었습니다. [센리히+ 2015]

(이전에는 논문에서 단어 기반 토큰화를 사용했습니다.)

BPE는 GPT-2에 의해 사용되었습니다. [래드포드+ 2019]

기본 아이디어: **토크나이저를 원시 텍스트로** 훈련시켜 어휘를 자동으로 판별합니다.

직관: 일반적인 문자 시퀀스는 단일 토큰으로 표현되고, 드문 시퀀스는 다음과 같이 표현됩니다.
여러 가지 면에서 그렇다.

GPT-2 논문은 단어 기반 토큰화를 사용하여 텍스트를 초기 세그먼트로 나누고 원본을 실행했습니다.
각 세그먼트에 BPE 알고리즘을 적용합니다.

스케치: 각 바이트를 토큰으로 시작하여 가장 일반적인 인접 토큰 쌍을 차례로 병합합니다.

토크나이저 훈련

```
    문자열 = "모자쓴 고양이" # @inspect 문자열
    매개변수 = train_bpe(문자열, num_merges=3)
```

토크나이저 사용

이제 새로운 텍스트가 주어지면 이를 인코딩할 수 있습니다.

```
    토크나이저 = BPETokenizer(매개변수)
    문자열 = "빠른 갈색 여우" # @inspect 문자열
    인덱스 = tokenizer.encode(문자열) # @inspect 인덱스
    재구성된 문자열 = tokenizer.decode(인덱스) # @inspect 재구성된 문자열
    문자열 == 재구성된 문자열을 주장합니다 .
```

과제 1에서는 다음과 같은 방법으로 이를 더욱 심화시킬 것입니다.

- encode()는 현재 모든 병합을 반복합니다. 중요한 병합만 반복합니다.
- 특수 토큰(예: <|endoftext|>)을 감지하고 보존합니다.
- 사전 토큰화를 사용합니다(예: GPT-2 토크나이저 정규식).
- 가능한 한 빨리 구현해 보세요.

```
def train_bpe(문자열: str, num_merges: int) -> BPETokenizerParams: # @inspect 문자열, @inspect num_merges
    문자열 의 바이트 목록으로 시작합니다 .
    인덱스 = list(map(int, string.encode("utf-8"))) # @inspect 인덱스
    병합: dict[tuple[int, int], int] = {} # index1, index2 => 병합된 인덱스
    어휘: dict[int, bytes] = {x: bytes([x]) for x in range(256)} # 인덱스 -> 바이트

    i가 범위(num_merges) 에 있는 경우 :
        각 토큰 쌍의 발생 횟수를 세어보세요.
        카운트 = defaultdict(int)
        zip(indices, indices[1:]) 의 index1, index2 에 대해 : # 인접한 각 쌍에 대해
            counts[(index1, index2)] += 1 # @counts 검사

        가장 흔한 쌍을 찾아보세요.
        쌍 = max(counts, key=counts.get) # @쌍 검사
        index1, index2 = 쌍

        그 쌍을 합치세요.
        new_index = 256 + i # @new_index 검사
        merges[pair] = new_index # @inspect 병합
        어휘[새로운_인덱스] = 어휘[인덱스1] + 어휘[인덱스2] # @ 어휘 검사
        인덱스 = merge(인덱스, 쌍, new_index) # @인덱스 검사

    BPETokenizerParams(vocab=vocab, merges=merges)를 반환합니다 .


__name__ == "__main__" 인 경우 :
    기본()
```